

B 的 X 锁时被拒绝, 只能等待 T_1 释放 B 上的锁。 T_1 为验算再读 A 、 B , 这时读出的 B 仍是 100, 求和结果仍为 150, 即可重复读。 T_1 结束后释放 A 、 B 上的 S 锁, 此时 T_2 才能获得对 B 的 X 锁。

上述三级封锁协议的主要区别在于什么操作需要申请封锁, 以及何时释放锁(即持锁时间)。三级封锁协议可以总结为表 12.2。表中还指出了不同的封锁协议使事务达到的一致性级别是不同的, 封锁协议级别越高, 一致性程度越高。

表 12.2 不同级别的封锁协议和一致性保证

封锁协议	X 锁		S 锁		一致性保证			隔离性 级别保证
	操作结束 释放	事务结束 释放	操作结束 释放	事务结束 释放	不丢失 修改	不脏读	可重 复读	
一级封锁协议		✓			✓			读未提交
二级封锁协议		✓	✓		✓	✓		读已提交
三级封锁协议		✓		✓	✓	✓	✓	可串行化

12.5 活锁和死锁

和操作系统一样, 封锁的方法可能引起活锁(livelock)和死锁(deadlock)等问题。

12.5.1 活锁

如果事务 T_1 封锁了数据 R , 事务 T_2 又请求封锁 R , 于是 T_2 等待; T_3 也请求封锁 R , 当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求, T_2 仍然等待; 然后 T_4 又请求封锁 R , 当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求…… T_2 有可能永远等待, 这就是活锁的情形, 如图 12.6(a) 所示。

避免活锁的简单方法是采用先来先服务的策略。当多个事务请求封锁同一数据对象时, 封锁子系统按请求封锁的先后次序对事务排队, 数据对象上的锁一旦释放就批准申请队列中第一个事务获得锁。

12.5.2 死锁

如果事务 T_1 封锁了数据 R_1 , T_2 封锁了数据 R_2 , 然后 T_1 又请求封锁 R_2 , 因 T_2 已封锁了 R_2 , 于是 T_1 等待 T_2 释放 R_2 上的锁; 接着 T_2 又申请封锁 R_1 , 因 T_1 已封锁了 R_1 , T_2 也只能等待 T_1 释放 R_1 上的锁。这样就出现了 T_1 在等待 T_2 , 而 T_2 又在等待 T_1 的局面, T_1 和 T_2 两个事务永远不能结束, 形成死锁。如图 12.6(b) 所示。

死锁的问题在操作系统和一般并行处理中已做了深入研究。目前在数据库中解决死锁问题主要有两类方法: 一类方法是采取一定措施来预防死锁的发生; 另一类方法是允许发生死锁, 采用一定手段诊断系统中有无死锁, 若有则进行解除。

T ₁	T ₂	T ₃	T ₄	T ₁	T ₂
LOCK R	⋮	⋮	⋮	LOCK R ₁	⋮
⋮	LOCK R	⋮	⋮	⋮	LOCK R ₂
⋮	等待	LOCK R	LOCK R	LOCK R ₂	⋮
UNLOCK R	等待	⋮	等待	等待	⋮
⋮	等待	LOCK R	等待	等待	LOCK R ₁
⋮	等待	⋮	等待	等待	等待
⋮	等待	UNLOCK	LOCK R	等待	等待
⋮	等待	⋮	⋮	等待	⋮

(a) 活锁

(b) 死锁

图 12.6 死锁与活锁示例

1. 死锁的预防

在数据库中，产生死锁的原因是两个或多个事务都已封锁了一些数据对象，然后又都请求对已被其他事务封锁的数据对象加锁，从而出现死等待。防止死锁的发生，其实就是要破坏产生死锁的条件。预防死锁通常有以下两种方法。

(1) 一次封锁法

一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行。例如图 12.6(b) 的例子中，如果事务 T₁ 将数据对象 R₁ 和 R₂ 一次加锁，T₁ 就可以执行下去，而 T₂ 等待。T₁ 执行完后释放 R₁、R₂ 上的锁，T₂ 继续执行。这样就不会发生死锁。

一次封锁法虽然可以有效地防止死锁的发生，但也存在如下问题：

① 一次就将以后要用到的全部数据加锁，势必扩大了封锁的范围，从而降低了系统的并发度。

② 数据库中的数据是不断变化的，原来不要求封锁的数据在执行过程中可能会变成封锁对象，所以很难事先精确地确定每个事务所要封锁的数据对象。为此只能扩大封锁范围，将事务在执行过程中可能要封锁的数据对象全部加锁，这就进一步降低了并发度。

(2) 顺序封锁法

顺序封锁法是预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实施封锁。例如在 B 树结构的索引中，可规定封锁的顺序必须是从根结点开始，然后是下一级的子结点，逐级封锁。

顺序封锁法可以有效地防止死锁，但也同样存在问题：

① 数据库系统中封锁的数据对象极多，并且随数据的插入、删除等操作而不断地变化，要维护这样的资源的封锁顺序非常困难，成本很高。

② 事务的封锁请求可以随着事务的执行而动态地决定, 很难事先确定每一个事务要封锁哪些对象, 因此也就很难按规定的顺序去施加封锁。

可见, 在操作系统中广为采用的预防死锁的策略并不太适合数据库的特点, 因此数据库管理系统在解决死锁的问题上普遍采用的是诊断并解除死锁的方法。

2. 死锁的诊断与解除

数据库系统中诊断死锁的方法与操作系统类似, 一般使用超时法或事务等待图法。

(1) 超时法

如果一个事务的等待时间超过了规定的时限, 就认为发生了死锁。超时法实现简单, 但其不足也很明显, 一是有可能误判死锁, 如事务因为其他原因而使等待时间超过时限, 系统会误认为发生了死锁; 二是时限若设置得太长, 死锁发生后不能及时发现。

(2) 事务等待图法

事务等待图是一个有向图 $G=(T,U)$ 。其中 T 为结点的集合, 每个结点表示正在运行的事务; U 为边的集合, 每条边表示事务等待的情况。若 T_1 等待 T_2 , 则在 T_1 、 T_2 之间画一条有向边, 从 T_1 指向 T_2 , 如图 12.7 所示。

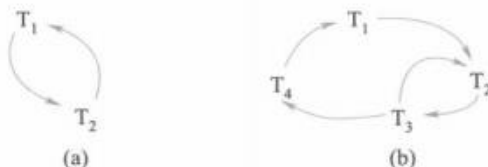


图 12.7 事务等待图示例

事务等待图动态地反映了所有事务的等待情况。并发控制子系统周期性地(比如每隔数秒)生成事务等待图并进行检测, 如果发现图中存在回路, 则表示系统中出现了死锁。

图 12.7(a)表示事务 T_1 等待 T_2 , T_2 又等待 T_1 , 产生了死锁。图 12.7(b)表示事务 T_1 等待 T_2 , T_2 等待 T_3 , T_3 等待 T_4 , T_4 又等待 T_1 , 产生了死锁。

当然, 死锁的情况可以多种多样。例如, 图 12.7(b)中事务 T_3 可能还等待 T_2 , 在大回路中又有小的回路。这些情况人们都已做了很深入的研究。

数据库管理系统的并发控制子系统一旦检测到系统中存在死锁, 就要设法解除。通常采用的方法是选择一个处理死锁代价最小的事务, 将其撤销, 释放此事务持有的所有锁, 使其他事务得以继续运行下去。当然, 对撤销的事务所执行的数据修改操作必须加以恢复。

12.6 并发调度的可串行性

数据库管理系统对并发事务不同的调度可能会产生不同的结果, 那么什么样的调度是正确的呢? 显然, 串行调度是正确的。执行结果等价于串行调度的调度也是正确的。这样的调度称为可串行化调度。